Unit-2: Conditional and looping statements

Main Topic	Subtopics	What You Learn
1. Conditional Statements	if statement	Execute block when condition is true
	if-else statement	Choose between two blocks
	else-if ladder	Multiple condition checking
	Nested if	if inside another if
	switch statement	Multi-way branching
	case label	Specific value check
	default case	Executed when no case matches
2. Looping Statements	for loop	Loop with initialization, condition, update
	while loop	Loop executed while condition is true
	do-while loop	Executes at least once
	Enhanced for loop (for-each)	Iterating arrays or collections
3. Loop Control Statements	break statement	Terminates loop immediately
	continue statement	Skips current iteration
	Labelled break	Break specific loop in nested loops
	Labelled continue	Continue specific loop

Unit-2: Conditional Statements (Java):

Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java, which provides a smooth flow of program.

Types of control flow statements:

1. Conditional statements
2. Loop statements
3. Jump statements

1. if Statement

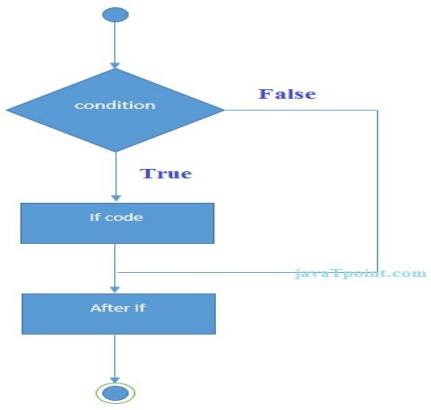
Definition : The **if statement executes a block of code only when the condition is true**.

If the condition is false, the block is skipped.

Syntax

```
if(condition) {
   // code executed if condition is true
}
```

Flow Diagram



Example

```
class IfExample {
   public static void main(String[] args)
      int age = 20;
      if(age >= 18) {
```

```
        System.out.println("Eligible to vote");
      }
   }
}
```
Output: Eligible to vote

Practical Example

ATM system:
```
if(balance > withdrawalAmount) {
   withdrawMoney();
}
```

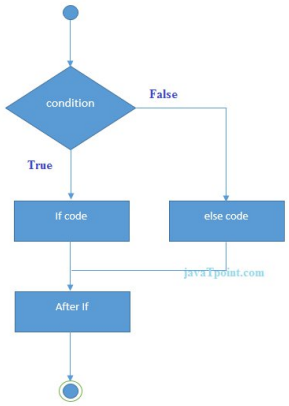
2. if-else Statement

Definition : The **if-else statement allows choosing between two blocks of code**.

- One block executes if condition is **true**
- Another block executes if condition is **false**

Syntax
```
if(condition) {
   // executes if true
}
else {
   // executes if false
}
```

Flow Diagram



Example
```
class IfElseExample {
   public static void main(String[] args) {
      int number = 7;
      if(number % 2 == 0) { System.out.println("Even Number");
      }
      else {
         System.out.println("Odd Number");
      }
   }
}
```
Output: Odd Number

3. if-else-if Ladder

Definition : Used when **multiple conditions need to be checked**.

The program evaluates conditions from **top to bottom**.

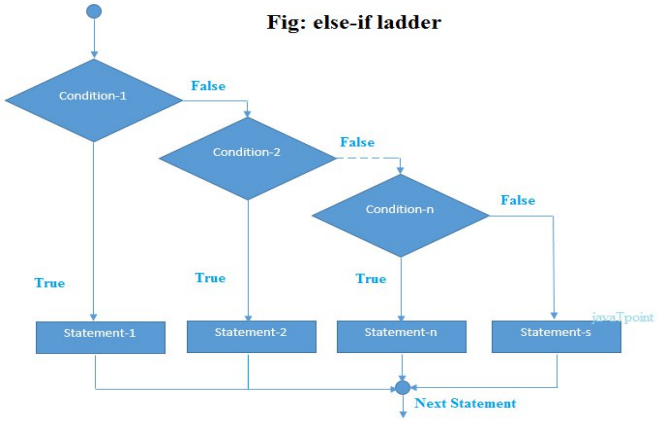
The first condition that is **true** executes its block.

Syntax

dhruvpatel16120

```
if(condition1) {
   // block1
}
else if(condition2) {
   // block2
}
else if(condition3) {
   // block3
}
else {
   // default block
}
```

Flow:



Example

```
class GradeExample {
  public static void main(String[] args) {
      int marks = 75;
      if(marks >= 90) {
         System.out.println("Grade A");
      }
      else if(marks >= 70) {
         System.out.println("Grade B");
      }
      else if(marks >= 50) {
         System.out.println("Grade C");
      }
      else {
         System.out.println("Fail");
      }
   }
}
```

Output: Grade B

4. Nested if Statement

Definition: A **nested if statement is an if statement inside another if statement**.
Used when **multiple conditions must be satisfied**.

Syntax

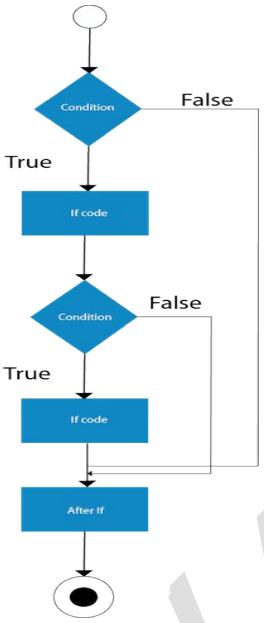
dhruvpatel16120

```
if(condition1) {
   if(condition2) {
      // execute block
   }
}
```

Example

```
class NestedIfExample {
   public static void main(String[] args) {
      int age = 25;
      boolean hasLicense = true;
      if(age >= 18) {
         if(hasLicense) {
            System.out.println("Allowed to drive");
         }
      }
   }
}
```

Output: Allowed to drive



Real-World Example

Online exam system:

```
if(studentRegistered) {
   if(paymentDone) {
      allowExam();
   }
}
```

5. switch Statement

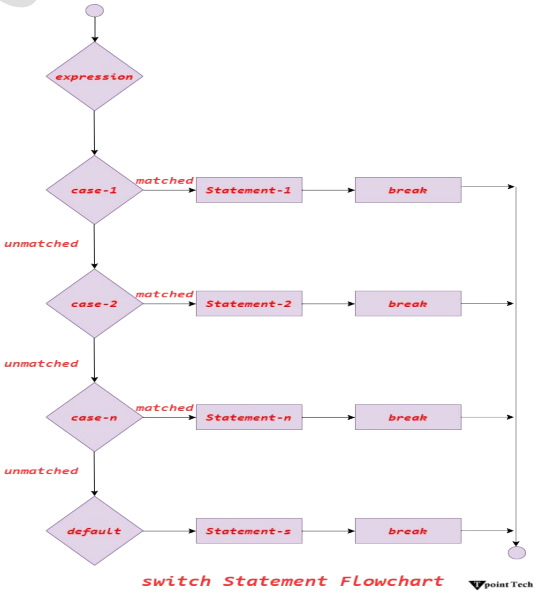
Definition : The **switch statement is used for multi-way branching**.

Instead of many if-else conditions, switch checks **multiple values of a variable**.

Syntax

```
switch(expression) {
   case value1:
      // code
      break;
   case value2:
      // code
      break;
   default:
      // default code
}
```

Flow



6. case Label : A **case label represents a specific value** that the expression can match.

Example:

```
case 1:
case 2:
case 3:
```

Example Program

```
class SwitchExample {
   public static void main(String[] args) {
```

dhruvpatel16120

```
    int day = 3;
    switch(day) {
      case 1:
        System.out.println("Monday");
        break;
      case 2:
        System.out.println("Tuesday");
        break;
      case 3:
        System.out.println("Wednesday");
        break;
      default:
        System.out.println("Invalid Day");
    }
  }
}
```

Output : Wednesday

7. default Case: The **default case executes when none of the cases match**.

Example:

```
default:
  System.out.println("Invalid Input");
```

Example

```
switch(choice) {
case 1:
  deposit();
  break;
case 2:
  withdraw();
  break;
default:
  System.out.println("Invalid choice");
}
```

Difference: if-else vs switch

Feature	if-else	switch
Conditions	Complex conditions	Exact value matching
Data types	Any boolean expression	int, char, String
Speed	Slower for many conditions	Faster for many cases
Usage	Logical decisions	Menu-based programs

Quick Revision Table

Statement	Purpose
if	Execute block if condition true
if-else	Choose between two blocks
else-if ladder	Multiple conditions
Nested if	Condition inside condition
switch	Multi-way branching
case	Specific value
default	When no case matches

Ternary Operator

We can also use ternary operator (? :) to perform the task of if...else statement. It is a shorthand way to check the condition. If the condition is true, the result of ? is returned. But, if the condition is false, the result of : is returned.

Syntax : (condition)? Result1:Result2;

Give Result1 if condition True else give Result2 for false.

jaypatel16120

Example

```
public class Main {
    public static void main(String[] args) {
      int number=13;
      //Using ternary operator
      String output=(number%2==0)?"even number":"odd number";
      System.out.println(output);
    }
}
```

Looping Statements:

Looping statements allow a program to **repeat a block of code multiple times** until a condition is satisfied.
They are part of **Unit-2: Conditional and Looping Statements** in the syllabus.
Looping is essential for:

- processing arrays
- repeating calculations
- iteration in algorithms

Overview of Looping Statements

Loop Type	Purpose	When to Use
for loop	Known number of iterations	Counting loops
while loop	Condition-based loop	Unknown iteration count
do-while loop	Executes at least once	Menu systems
Enhanced for loop	Traversing arrays/collections	Read-only iteration

1. for Loop

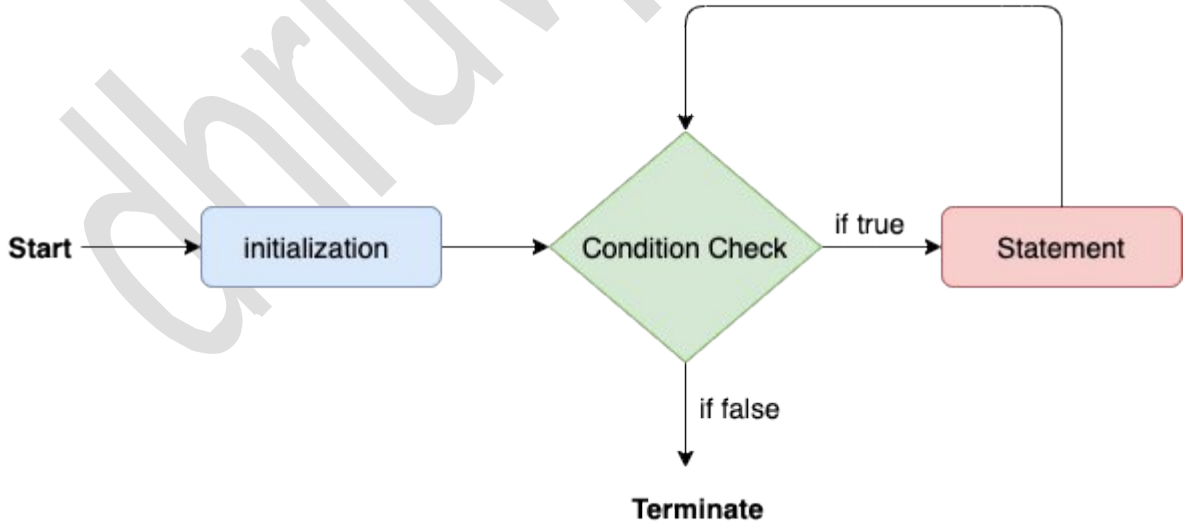
Definition: The **for loop repeats a block of code a specific number of times**.
It contains three parts:

1. Initialization
2. Condition
3. Update

Syntax

```
for(initialization; condition; update) {
  // loop body
}
```

Flow Diagram



Example

```
class ForExample {
  public static void main(String[] args) {
    for(int i = 1; i <= 5; i++) {
```

dhruvpatel16120

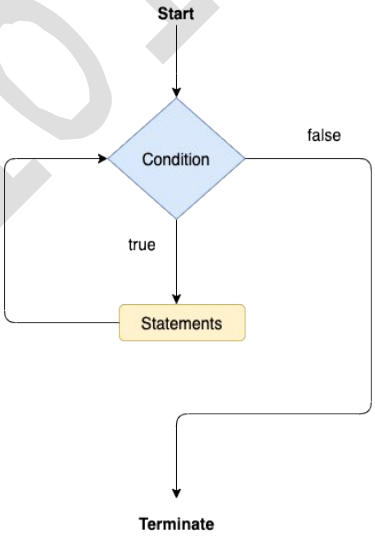
```
        System.out.println(i);
      } } }
```
Output
1
2
3
4
5

Real Example
Print multiplication table:
```
for(int i=1; i<=10; i++) {
   System.out.println("5 X "+ i + "=" +5 * i);
}
```

2. while Loop

Definition The **while loop executes a block of code repeatedly as long as the condition is true**.
The condition is checked **before execution**.

Syntax
```
while(condition) {
   // loop body
}
```
Example
```
class WhileExample {
   public static void main(String[] args) {
      int i = 1;
      while(i <= 5) {
         System.out.println(i);
         i++;  // essential to stop the loop at 5
      }
   }
}
```
Output
1
2
3
4
5



Practical Example
Password validation:
```
while(!passwordCorrect) {
   askPassword();
}
```

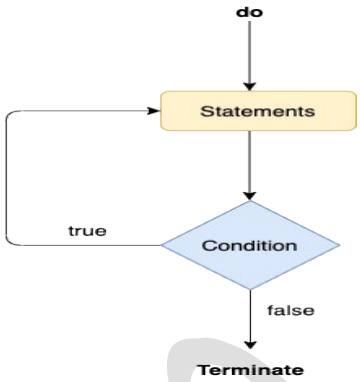
3. do-while Loop

Definition: The **do-while loop executes the loop body at least once**, even if the condition is false.
Condition is checked **after execution**.

dhruvpatel16120

Syntax

```
do {
   // loop body
} while(condition);
```



Example

```
public class Main {
      public static void main(String[] args) {
      int i = 1;
      do {
         System.out.println(i);
         i++;
         System.out.println("loop executed");
      } while(i <= 1);
      System.out.println(i);
   }
}
```

Output

```
1
loop executed
2
```

Real Example

Menu-driven system:

```
do {
  showMenu();
  choice = getChoice();
} while(choice != 0);
```

Difference Between while and do-while

Feature	while loop	do-while loop
Condition check	Before loop execution	After loop execution
Minimum execution	0 times	At least 1 time
Syntax	while(condition)	do { } while(condition);
Type	Entry-control loop	Exit-Control loop

4. Enhanced for Loop (for-each)

Definition: The **enhanced for loop is used to iterate through arrays or collections easily**.
It simplifies reading elements.

Syntax

```
for(datatype variable : array) {
   // use variable
}
```

Example

```
class ForEachExample {
   public static void main(String[] args) {
      int numbers[] = {10,20,30,40};
      for(int num : numbers) {
         System.out.println(num);
      }
   }
}
```

Output

```
10
20
30
```

dhruvpatel16120

Real Example

Processing student marks:

```
int marks[] = {70,80,90};
for(int m : marks) {
   System.out.println(m);
}
```

Difference Between for and Enhanced for

Feature	for loop	Enhanced for
Access index	Yes	No
Simplicity	More complex	Simpler
Use case	General loops	Arrays/collections

Example Program Using All Loops

```
class LoopExample {
   public static void main(String[] args) {
      // for loop
      for(int i=1;i<=3;i++) {
         System.out.println("For: " + i);
      }
      // while loop
      int j=1;
      while(j<=3) {
         System.out.println("While: " + j);
         j++;
      }
      // do-while
      int k=1;
      do {
         System.out.println("DoWhile: " + k);
         k++;
      } while(k<=3);
     System.out.println(“value of K after loop: ” + k);
   }
}
```

Most Expected Mid-Sem Questions

Theory

1. Explain **for loop in Java**.
2. Difference between **while and do-while**.
3. Explain **enhanced for loop**.

Programming Questions

Examples teachers ask:

- Print **1 to 10 using for loop**
- Print **even numbers using while**
- Print **multiplication table**
- Traverse **array using for-each loop**

Quick Revision Table

Loop	Condition Check	Usage
for	Before execution	Fixed iterations
while	Before execution	Unknown iterations
do-while	After execution	At least one run
for-each	For arrays/collections	Iteration

dhrupatel16120

Loop Control Statements

Loop control statements modify the **normal execution of loops**.
They allow a program to **terminate a loop early, skip iterations, or control nested loops**.
These statements appear in **Unit-2: Conditional and Looping Statements**.

Overview of Loop Control Statements

Statement	Purpose
break	Immediately terminate loop
continue	Skip current iteration
labelled break	Exit specific outer loop
labelled continue	Skip iteration of outer loop

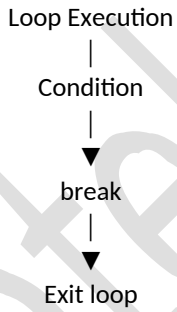
1. break Statement

Definition: The **break statement immediately terminates the loop** and transfers control to the next statement after the loop.
Used in:

- loops
- switch statements

Syntax break;

Flow



Example

```
class BreakExample {
   public static void main(String[] args) {
      for(int i=1; i<=10; i++) {
         if(i == 5) {
            break;
         }
         System.out.println(i);
      }
   }
```

Output
1
2
3
4
Loop stops when i = 5.

Real Example
Search operation:

```
for(int i=0;i<array.length;i++) {
   if(array[i] == target) {
      break;
   }
}
```

Once element is found, loop stops.

dhruvpatel16120

2. continue Statement

Definition: The **continue statement skips the current iteration and moves to the next iteration of the loop**.
It does not terminate the loop.
Syntax : continue;

Flow

Loop iteration
|
Condition check
|
continue
|
Skip remaining statements
|
Next iteration

Example

```
class ContinueExample {
   public static void main(String[] args) {
      for(int i=1; i<=5; i++) {
         if(i == 3) {
            continue;
         }
         System.out.println(i);
      }
   }
}
```

Output
1
2
4
5
Number 3 is skipped.

Real Example
Skip invalid values:

```
for(int i=1;i<=10;i++) {
   if(i % 2 == 0)
      continue;
   System.out.println(i);
}
```

Prints only odd numbers.

Difference Between break and continue

Feature	break	continue
Loop behavior	Terminates loop	Skips iteration
Execution	Loop stops	Loop continues
Usage	Exit loop early	Skip unwanted values

3. Labelled break

Definition: A **labelled break terminates a specific outer loop in nested loops**.
Normally break exits only the **innermost loop**.
Labelled break exits the **specified loop**.

Syntax
labelName:

dhruvpatel16120

```
for(...) {
   for(...) {
      break labelName;
   }
}
```

Example

```
class LabelBreakExample {
   public static void main(String[] args) {
      outer:
      for(int i=1; i<=3; i++) {
         for(int j=1; j<=3; j++) {
            if(i==2 && j==2) {
               break outer;
            }
            System.out.println(i + " " + j);
         }
      }
   }
}
```

Output

```
1 1
1 2
1 3
2 1
```

When condition becomes true, **both loops stop**.

4. Labelled continue

Definition : A **labelled continue skips the current iteration of a specified outer loop**.
Control moves to the **next iteration of the labelled loop**.

Syntax

```
labelName:
for(...) {
   for(...) {
      continue labelName;
   }
}
```

Example

```
class LabelContinueExample {
   public static void main(String[] args) {
      outer:
      for(int i=1; i<=3; i++) {
         for(int j=1; j<=3; j++) {
            if(j==2) {
               continue outer;
}
          System.out.println(i + " " + j);
         }
      }
   }
}
```

Output

```
1 1
2 1
3 1
```

dhruvpatel16120

When j==2, control jumps to next iteration of outer loop.

Real-World Example

Nested loops used in matrix processing:

```
outer:
for(int i=0;i< rows;i++) {
   for(int j=0;j< cols;j++) {
      if(matrix[i][j] == 0)
         continue outer;
   }
}
```

Skip rows containing zero.

dhruvpatel16120